vLLM Neuron Beta - May 26, 2026

Overview

Please note: **THIS IS BETA SOFTWARE. PLEASE SHARE YOUR FEEDBACK AND EXPERIENCE ONLY WITH NEURON TEAM FOR IMPROVEMENTS.**

This guide walks through running the Llama3.2 1B and GPT-OSS 20B models on AWS Trainium (Trn2) using the vllm-neuron private beta container image. The beta image is based on the Neuron 2.30 SDK release.

What's new in this beta release:

- First release of the enhanced vLLM Neuron plugin for vLLM
- Functional support for Llama3 and GPT-OSS models on AWS Trainium (Trn2 and Trn3). Compatible checkpoints include:
  - meta-llama/Llama-3.2-1B-Instruct
  - meta-llama/Llama-3.1-8B-Instruct
  - meta-llama/Llama-3.3-70B-Instruct
  - openai/gpt-oss-20b
  - openai/gpt-oss-120b

Prerequisites

- Launch a Trn2 or Trn3 instance. Note the container and instructions below are tested on `trn2.48xlarge` instance (16 Neuron devices), however the container should work on any Trn2 or Trn3 instance
- Docker installed

Step 1 — Pull the Image

> *export IMAGE=421672808698.dkr.ecr.us-east-1.amazonaws.com/concourse-release-28ce3c3:latest*
>
> *aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 421672808698.dkr.ecr.us-east-1.amazonaws.com*
>
> *docker pull "$IMAGE"*

Step 2 — Install the Neuron Driver

The container image include a Neuron driver. Extract and install it on the host:

```
# Extract driver RPM from the container image
TMP=$(mktemp -d)
docker create --name extract-driver "$IMAGE"
docker cp extract-driver:/opt/aws/neuron/driver/. "$TMP/"
docker rm extract-driver

# Install on Ubuntu/Debian host
sudo dpkg -i $TMP/aws-neuronx-dkms_*.deb
```

```
# Verify
modinfo neuron | grep "^version"
```

Step 3 — Start the Container

```
sudo docker run -d --privileged \
  --name vllm_neuron \
  -v /path/to/hf_cache:/root/.cache/huggingface \
  -v /tmp/vllm_neff_cache:/root/.cache/vllm \
  -v /tmp/vllm_cache:/tmp/vllm_cache \
  -v /tmp/nki_cache:/tmp/nki_cache \
  --env "HF_TOKEN=$HF_TOKEN" \
  -p 8000:8000 \
  --ipc=host \
  --device /dev/neuron0 --device /dev/neuron1 --device /dev/neuron2 --device /dev/neuron3 \
  --device /dev/neuron4 --device /dev/neuron5 --device /dev/neuron6 --device /dev/neuron7 \
  --device /dev/neuron8 --device /dev/neuron9 --device /dev/neuron10 --device /dev/neuron11 \
  --device /dev/neuron12 --device /dev/neuron13 --device /dev/neuron14 --device /dev/neuron15 \
  "$IMAGE" \
  sleep infinity
```

Notes:

- `-v /path/to/hf_cache:/root/.cache/huggingface` caches the model weights so subsequent runs skip the download.
- `-v /tmp/vllm_neff_cache:/root/.cache/vllm` persists the compiled Neuron NEFFs. with this mount, subsequent runs reuse them and skip recompilation.
- All 16 Neuron devices must be passed explicitly with `--device`.
- `-p 8000:8000` and `--ipc=host` enable you to send requests to a vLLM server running inside the container.
- `--env "NEURON_SKIP_EFA_AFFINITY=1"` is required on instances without EFA configured, including 3xl instances.

Step 4 — Run a Sample Server

Open a shell in the container.

```
docker exec -it <container-id> /bin/bash
```

LLAMA3.2 1B

Note: Llama3 checkpoints are gated on HuggingFace, so you must use a HuggingFace account with access to run these scripts with a model ID. You can provide your HuggingFace token via the `HF_TOKEN` env var. Alternatively, you can provide the local path to a Llama checkpoint instead of a model ID.

```
vllm serve meta-llama/Llama-3.2-1B-Instruct \
  --max-model-len 10240 \
  --max-num-batched-tokens 2048 \
  --max-num-seqs 4 \
  --tensor-parallel-size 8 \
  --additional-config '{
    "neuron_config": {
      "num_batched_tokens_buckets": [2048],
```

```
      "num_seqs_buckets": [4]
    }
  }'
```

After the server starts up, send a request.

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
        "model": "meta-llama/Llama-3.2-1B-Instruct",
        "messages": [
            {
                "role": "user",
                "content": "Hello"
            }
        ]
    }'
```

GPT-OSS 20B

```
vllm serve openai/gpt-oss-20b \
  --max-model-len 10240 \
  --max-num-batched-tokens 2048 \
  --max-num-seqs 2 \
  --tensor-parallel-size 8 \
  --hf-overrides '{"quantization_config": {}}' \
  --additional-config '{
    "neuron_config": {
      "num_batched_tokens_buckets": [2048],
      "num_seqs_buckets": [2]
    }
  }'
```

After the server starts up, send a request.

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
        "model": "openai/gpt-oss-20b",
        "messages": [
            {
                "role": "user",
                "content": "Hello"
            }
        ]
    }'
```

Step 5 - Review the SDK Documentation

This beta includes the vLLM Neuron SDK documentation attached as a PDF. This documentation includes additional tutorials and details about supported models and features. Note that the instructions in the PDF are tailored for usage with a Neuron DLAMI, which isn't part of this beta, so certain instructions need minor adjustments to work with the beta image.

Step 6 - Review the Source Code

If you would like to review the source code of the plugin and Llama3 and GPT-OSS model implementations, you can find the source code in the container at the following location.

```
/workspace/vllm_neuron
```

Troubleshooting

`NRT_FAILURE IN NRT_INIT()` / `LOGICAL NEURON CORE(S) NOT AVAILABLE`

The host Neuron driver doesn't match the container's runtime. Install the driver baked into the image (Step 1).

COMPILER SCRATCH DIRECTORIES ACCUMULATING IN CWD

After running, you may see many short hex-named directories alongside `log-neuron-cc.txt` and `global_metric_store.json` in the container's working directory:

```
/workspace/
├── 12db1dcdf9b8269b/
│   └── neuronxcc.private_nkl.transpose.tiled_dve_transpose_10_lnc0_*.json
├── 3762b8b28236e9a0/
├── ...
├── log-neuron-cc.txt
├── global_metric_store.json
├── vllm_neuron/
```

These are NKI kernel scratch directories written by `neuronx-cc` directly into `os.getcwd()`. They are not the NEFF cache (which lives under `/root/.cache/vllm/<32-char-hash>/`) and are safe to delete.

IDENTIFYING NEFF CACHE ENTRIES

Each compiled NEFF is stored under the cache directory (`/root/.cache/vllm/<hash>/` inside the container, or `/tmp/vllm_neff_cache/<hash>/` on the host with the volume mount from Step 3). To identify which model component a NEFF corresponds to, inspect example_inputs.txt:

```
# Inside the container:
for dir in /root/.cache/vllm/*/; do
  key=$(basename "$dir")
  shape=$(grep -A1 "Input 0:" "$dir/example_inputs.txt" | grep "Shape:" | head -1 | sed
's/.*Shape: //')
  size=$(du -sh "$dir/graph.neff" | cut -f1)
  echo "$key | $size | input0=$shape"
done
```

Known Limitations

- GPT-OSS 120B on Trn3 with MxFP4 dtype has a known accuracy issue that will be fixed in a future release.
- To use GPT-OSS with prefix caching, use a TP8 server config (`--tensor_parallel_size=8`). Prefix caching is enabled by default.
- Prefix caching is accurate only when the server uses segmented prefill. Segmented prefill is enabled by default, but can be unexpectedly disabled at low `--max-model-len` or if you set `--max-num-batched-tokens` to equal `--max-model-`

`len`.

- `--env "NEURON_SKIP_EFA_AFFINITY=1"` is required on instances without EFA configured, including 3xl instances.

Feedback and Support

Please report issues through your AWS Account team during this private beta.

Beta feedback survey:
https://pulse.amazon/survey/RLJLQBCZ